

答辩提纲

重点演示用例：UC01 注册登录、UC02 搜索、UC04 下单支付发货收货。最终范围 UC01-UC12。

给 PPT 的建议页与数字

数字来自已提交报告，制作幻灯片时直接抄，不要口算。

1. 项目：校园购物 + 二手；仓库 <https://github.com/tchen-0213/softw>
2. 原系统：React → 一个 Express → MySQL shopping_platform；标签 monolith-start (10fa639)
3. 改造：Docker 三容器、GitHub Actions、Kubernetes YAML、三个业务微服务 + 网关；标签 microservices-v1 (63585e0)
4. 划分：user / product / order；库 softw_users / softw_catalog / softw_orders；网关不算业务服务；图 33-MS-SPLIT
5. 容器：单体 mysql + backend + frontend (8080 / 3001)；微服务另起一套（前端 8082、网关 8081）
6. CI：工作流 softw-ci-cd，测试失败不构建镜像；已验证绿勾 [#77](#)
7. 测试 (2026-09-03)：后端 120/120、前端 100/100、API 32/32、E2E 单体与微服务均 42/42、网关 API 15/15
8. 压测（三接口各 3 次）：列表吞吐 +61.9%、延迟 -38.6%、CPU +55.2%；
不要写微服务天然更快
9. HPA：1 → 3 → 5 → 1；故障：商品 503、订单 206，其他服务仍存活
10. 账号：demo-seller@example.com / Demo@123456；本地用 http://localhost:8080

成品 .pptx 放在本目录。技术总结见 技术总结报告.md。

4 分钟项目与架构说明

1. 项目背景：购物与二手交易平台。
2. 原系统：React/Vite 前端、Express 后端、MySQL。
3. 小学期改造：容器化、CI/CD、Kubernetes、微服务拆分。
4. 服务划分：网关、用户服务、商品服务、订单服务。

7 分钟现场演示

1. 运行 `npm run verify` 展示测试和构建。
2. 运行 `npm run compose:up` 展示单体容器化。
3. 查看 `/api/health`。
4. 应用 `03_devops/k8s/microservices` 展示网关和 3 个业务服务。
5. 访问 `api-gateway /health`、`/version` 和一个业务接口。
6. 触发 `product-service` 缩容为 0，访问订单依赖健康接口展示降级。
7. 展示 HPA 配置和压测结论表（不要现场重跑 18 组 unless 助教要求）。

4 分钟问答准备

- 为什么这样拆服务（身份 / 标的 / 交易过程，不是 12 个用例 12 个服务）。
- 每张表归谁管理；评价和聊天现在在商品服务。
- 测试失败为什么会阻断部署。
- 微服务查询为什么可能更快、资源却更贵。
- 本人负责的任务、提交和证据。